Page 1 – Fact Table vs Dimension Table

1. Fact Table – Key Points

- Stores measurable / quantitative metrics (facts)
- Each row represents a business event at a defined grain
- Contains foreign keys to dimension tables
- Usually very large in size
- Mostly numeric measures, but can also include transaction id, date key, flags, degenerate dimensions
- Example facts: sales_amount, quantity, discount
- **Factless fact tables** also exist (contain only keys, no numeric measures)

Example: Fact Sales

date_key	customer_key	product_key	sales_amount	quantity	discount
20240101	1001	501	250.00	5	10.00
20240101	1002	502	120.00	2	0.00
20240102	1001	501	300.00	3	15.00
20240102	1003	503	500.00	10	25.00
...	...	...	...	...	...

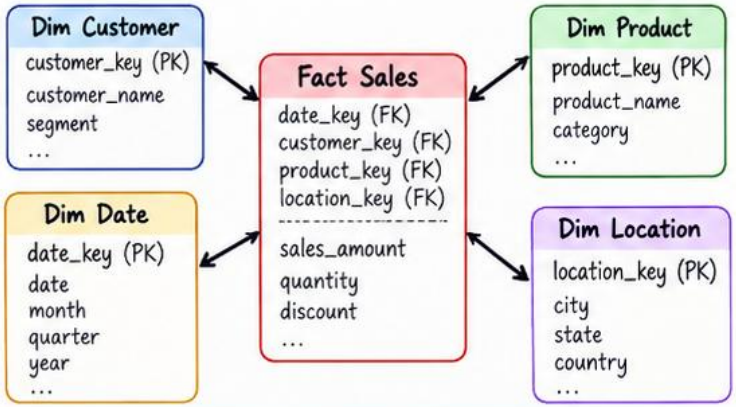
2. Dimension Table – Key Points

- Stores descriptive attributes that give business context
- Explains who, what, where, when, how
- Usually smaller than fact tables
- Contains a primary key (often surrogate key)
- Can include text, numeric attributes, dates, flags
- Supports hierarchies, e.g. Country → State → City
- Surrogate keys are recommended, especially for SCD Type 2

Example: Dim Customer

customer_key	customer_name	segment	country
1001	Acme Corp	Retail	USA
1002	Beta Ltd	Corporate	USA
1003	Gamma Inc	SMB	Canada
1004	Delta Co	Enterprise	UK
...	...	...	...

3. How Fact and Dimension Work Together



- Fact table is at the center of the star schema, and dimension tables are around it.
- They are most valuable together because fact gives measurable values and dimension gives descriptive context for analysis.

4. Fact vs Dimension – Comparison Table

Aspect	Fact Table	Dimension Table
Purpose	quantify business events	describe business context
Data type	measures + FKs	descriptive attributes
Size	very large	smaller
Keys	foreign keys to dimensions	primary/surrogate key
Grain	defines level of detail	provides context at various levels
Hierarchies	usually not stored here	usually stored here
Usage	aggregations / analysis	filtering / grouping / slicing

5. Important Design Rules

1. Define **grain first.**
2. Keep **descriptive attributes** in dimensions.
3. Keep **measures** in facts.
4. Do not join fact to dimension on natural key when SCD Type 2 is used; prefer **surrogate key** in fact.
5. Facts and dimensions should be designed around a **business process.**

Good design leads to accurate analysis and better business decisions.

6. Types of Measures

Type	Description	Examples
Additive	Can be summed across all dimensions (time, product, customer etc.)	sales amount, quantity
Semi-additive	Can be summed across some dimensions (but not all, usually not time)	account balance, inventory level
Non-additive	Cannot be summed; must be calculated (usually ratios or percentages)	ratio, percentage, conversion rate

Page 2 - Grain, Fact Types, Star vs Snowflake, Data Lake vs Data Warehouse

① Types of Fact Tables

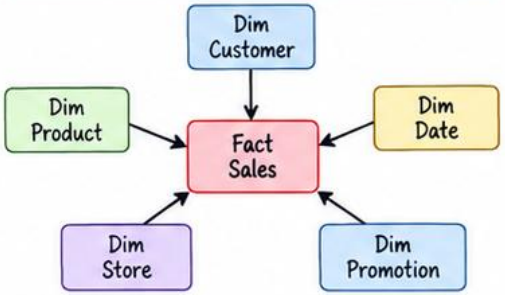
Type	What it represents	Example	Grain
Transaction Fact	one business transaction / event	sales order line	one row per order line
Periodic Snapshot	state at regular intervals	daily inventory balance	one row per product per day
Accumulating Snapshot	process with milestones	order placed → shipped → delivered	one row per order / workflow item
Factless Fact	event / relationship without numeric measure	student attendance / website click	one row per event

② What is Grain? (Most Important)

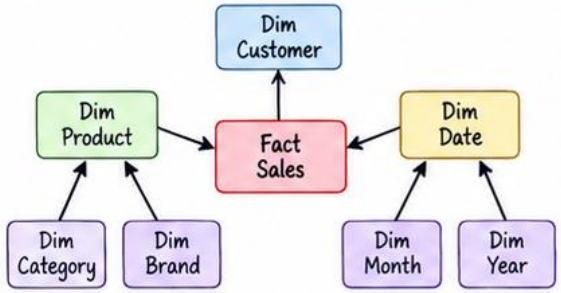
- Grain tells what one row in the fact table represents.
- Must be defined before selecting measures and dimensions.
- Examples:
  - one row per order line
  - one row per order
  - one row per customer per day
  - one row per product per warehouse per day
- Do not mix multiple grains in one fact table.

③ Star vs Snowflake Schema

Star Schema (Denormalized Dimensions)



Snowflake Schema (Normalized Dimensions)



Aspect	Star Schema	Snowflake Schema
Dimension structure	Denormalized dimensions (single tables)	Normalized dimensions (split into multiple tables)
Redundancy	Higher redundancy (duplicate data in dimension tables)	Less redundancy (minimizes duplicate data)
Number of joins	Fewer joins	More joins
Query simplicity	Simpler design and queries	More complex queries
Best use case	Preferred for most analytical reporting use cases	Useful for complex hierarchies or shared reference data

④ Data Lake vs Data Warehouse (vs Lakehouse)

Aspect	Data Lake	Data Warehouse	Data Lakehouse
Purpose	Flexible storage for all data	Governed analytical reporting	Combines lake flexibility with warehouse capabilities
Data types	Structured, semi-structured, unstructured (all types)	Mostly structured / modeled	All types (structured + semi-structured + unstructured)
Schema	Schema-on-read	Schema-on-write	Both (schema-on-read with governance / table formats)
Ingestion	Easy, high-volume, any format	Controlled, transformed, modeled	Supports batch and streaming
Storage cost	Lower cost	Higher cost	Cost-effective (depends on implementation)
Flexibility	Highly flexible	Less flexible	Flexible with governance
Use cases	Raw data, data science / ML, unknown future use cases	BI reporting, dashboards, consistent business metrics	Both BI and ML on same platform
Users	Data engineers, data scientists	Business analysts, BI teams	Both data engineers, data scientists and business users

⑤ When to Use What?

Use Data Lake when:
- Multiple formats (structured, semi-structured, unstructured)
- Raw data
- Data science / ML
- Unknown future use cases

Use Data Warehouse when:
- Mostly structured reporting data
- Consistent business metrics
- BI / dashboards

Use Lakehouse when:
- Both BI and ML on same platform
- Need open table formats
- Governed data

Page 3 – Top Scenario Based Interview Questions

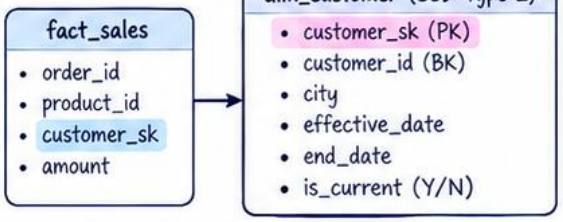
Fact vs Dimension, Grain, SCD, Star vs Snowflake, Data Lake vs DW

1 Historical vs Current Customer City (SCD Type 2)

Scenario:
Customer spent ₹50,000 in Delhi and later ₹30,000 in Bengaluru.
Need report by city at time of purchase and report by current city.

What interviewer expects:
- Use SCD Type 2
- Correct grain and keys
- Show how to get historical and current city reports

Key answer points:
- Use star schema
- Customer dimension as SCD Type 2 with surrogate key (customer_sk), business key (customer_id), effective dates, current flag (is_current)
- Fact grain = one row per order line / sales transaction
- Fact stores customer_sk, not just customer_id
- Historical city report joins fact to historical customer version using customer_sk
- Current city report resolves business key then joins to current version where is_current = Y
- Validate totals: Historical → Delhi 50k, Bengaluru 30k
  Current → Bengaluru 80k



2 Define Fact Table Grain & Handle Order-Level Shipping

Scenario:
One order has 3 products and one shipping fee.

What interviewer expects:
- Define correct grain
- How to handle order-level measures
- Avoid double counting

Key answer points:
- Prefer order-line grain for sales fact (one row per product in order).
- Keep shipping either in a separate order-level fact or allocate it to order lines using a clear rule.
- Do not repeat the shipping fee on every line to avoid revenue inflation.
- Returns should be modeled as a separate event, or use negative amount / return flag.

Order 1001		
product_id	qty	amount
P1	1	10,000
P2	1	15,000
P3	1	20,000

Shipping (order level)
500

↓

Keep shipping separate or allocate carefully (do not repeat on every line)

3 Inflated Revenue After Join

Scenario:
Fact shows ₹200 cr but after joining customer dimension, report shows ₹350 cr.

What interviewer expects:
- Check join logic and cardinality
- Identify root cause
- How to fix it

Key answer points:
- Check join keys and cardinality.
- Likely wrong join on customer_id instead of surrogate key with SCD Type 2 (multiple rows per customer).
- Compare row counts before and after join.
- Check for duplicate dimension rows / overlapping effective dates.
- Fix join logic (use customer_sk and correct date logic).
- Do not hide the issue with DISTINCT.

Example issue:
Joining on customer_id duplicates fact rows if multiple dimension versions exist.

customer_id	city	is_current
C001	Delhi	N
C001	Bengaluru	Y

4 Choose Correct Type of Fact Table

Scenario:
Need sales report, inventory balance, and order fulfillment tracking.

What interviewer expects:
- Map requirement to correct fact type
- Understand why separate facts are needed

Key answer points:
- Sales → Transaction fact (one row per order line / sale)
- Inventory → Periodic snapshot fact (e.g., daily balance per product per location)
- Order fulfillment → Accumulating snapshot fact (track order through multiple milestones)
- Use separate facts because business process and grain are different.
- Inventory is semi-additive across time (sum across products/location, not across time).

5 Star vs Snowflake Decision

Scenario:
50M products with Category → Subcategory → Brand → Product hierarchy, Dashboards need revenue by category/brand/product.

What interviewer expects:
- Recommend schema design
- Explain trade-offs
- When to use snowflake

Key answer points:
- Start with star schema for simplicity and BI performance.
- Consider snowflake if hierarchy is complex or shared reference data needs normalization.
- Trade-off: star = simpler, fewer joins, better performance; snowflake = less redundancy, more joins.
- If category/brand changes historically, use SCD where required.

6 Data Lake vs Data Warehouse vs Lakehouse

Scenario:
10 TB/day from JSON logs, SQL transactions, PDFs, images. Finance- needs dashboards, DS needs raw data, AI team needs document search.

What interviewer expects:
- Choose the right platform
- Show data organization and consumer-specific layers
- Mention key design considerations

Key answer points:
- Lakehouse is often the best fit for this use case.
- Ingest all data (structured, semi-structured, unstructured).
- Use layered architecture: Bronze (raw), Silver (cleaned/validated), Gold (business-ready).
- Provide raw data for DS/AI and curated gold data for BI dashboards.
- Include governance, schema checks, dedup, and consumer-specific layers.
- If only structured reporting is needed, a warehouse alone may be enough.

⚠ Common interviewer traps

1. Not defining grain
2. Joining on natural key instead of surrogate key for SCD2
3. Mixing multiple grains in one fact
4. Repeating order-level measures across rows
5. Assuming snowflake has more redundancy
6. Confusing lake and warehouse use cases

1. FACT TABLE vs DIMENSION TABLE

① Fact Table (Measures / Facts)

- Stores quantifiable, measurable metrics related to a business process
- Each row represents a business event at a defined grain (e.g. order line)
- Contains foreign keys to dimension tables
- Mostly numeric columns (measures)
- Usually very large in size
- Types: Transaction, Periodic Snapshot, Accumulating Snapshot, Factless Fact

② Dimension Table (Descriptive Attributes)

- Stores descriptive attributes about a business entity (who, what, where, when, how)
- Provides context to facts
- Has a primary key (surrogate key)
- Contains hierarchies (e.g. country → state → city)
- Attributes can be text, numeric, date, flags
- Supports SCD (Type 1 / Type 2)
- Usually smaller in size

Example: fact_sales (Order line level)

order_id	product_sk	customer_sk	date_sk	qty	amount
01	101	501	20250101	2	5000
01	102	501	20250101	1	2000
02	103	502	20250102	3	7500
...					

Example: dim_customer (SCD Type 2)

customer_sk	customer_id	name	start_date	end_date	is_current
501	C101	Rahul	2024-01-01	2024-06-30	N
502	C101	Bengalu	2024-07-01	9999-12-31	Y
...					

Common Measures

- Additive : sales_amount, quantity
- Semi-additive : account balance, inventory
- Non-additive : percentage, ratio, distinct count

Common Attributes

- Name, segment, city, region
- Date attributes (join to date dim)
- Product category, brand, hierarchy
- Employee, supplier, channel, location etc.

2. TYPES OF FACT TABLES & GRAIN

Type	What it represents	Example	Grain	Use case
Transaction Fact	Single business event	Sales order line	One row per order line	Most common (e-commerce, retail)
Periodic Snapshot	State at regular intervals	Daily inventory balance	One row per product per day	Inventory, account balance
Accumulating Snapshot	Process with multiple milestones	Order placed → shipped → delivered	One row per order	Order fulfillment, loan process
Factless Fact	Event/relationship without measure	Student attendance, website click	One row per event	When only occurrence matters

What is Grain?

- Defines what one row in fact table represents
- Must be clearly defined before selecting measures & dimensions
- Examples:
  - One row per order line (sales)
  - One row per customer per day (snapshot)
  - One row per product per warehouse per day (inventory)
  - One row per order (accumulating snapshot)

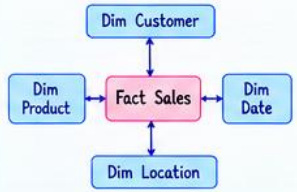
Key Points

- ✓ Choose correct fact type
- ✓ Keep grain consistent
- ✓ Use surrogate keys (esp. SCD)
- ✓ Maintain conformed dimensions
- ✓ Validate data quality and counts
- ✓ Handle late arriving data
- ✓ Avoid mixing multiple grains

3. STAR vs SNOWFLAKE SCHEMA

① Star Schema (Preferred for most use cases)

- Denormalized dimension tables
- Fact table at the centre
- Simpler design and queries
- Fewer joins → faster performance
- May contain redundant attributes
- Easier to understand and maintain



② Snowflake Schema

- Normalized dimension tables
- Dimension tables split into multiple related tables
- Less redundancy
- More joins and complex queries
- Useful for complex hierarchies
- Better for some maintenance use cases



Aspect	Star Schema	Snowflake Schema
Dimension structure	Denormalized	Normalized
Redundancy	More redundant attributes	Less redundancy
No. of joins	Fewer joins	More joins
Query performance	Usually faster	Can be slower (depends)
Complexity	Simple	More complex
Best use case	Analytical reporting	Complex hierarchies / shared

4. DATA LAKE vs DATA WAREHOUSE (vs LAKEHOUSE)

Aspect	Data Lake	Data Warehouse	Data Lakehouse (Best of both)
Purpose	Flexible storage for all data	Governed analytical reporting	Combines lake flexibility with DW capabilities
Data types	Structured, semi-structured, unstructured	Mostly structured	All types
Schema	Schema-on-read (after storing)	Schema-on-write (before storing)	Supports both (managed tables)
Ingestion	ELT (common)	ETL/ELT	ELT (common)
Storage cost	Typically cheaper (object storage)	Higher (optimized warehouse storage)	Optimized (open table formats like Delta/Iceberg)
Flexibility	High	Lower (schema changes take effort)	High with governance
Use cases	Data science, ML, new data, exploration	BI, operational analytics	Both BI and ML
Users	Data engineers, data scientists	Business analysts, operational users	All users

⑧ Data Lakehouse Architecture (Medallion)



5. SCENARIO 1 – SCD TYPE 2 (HISTORICAL vs CURRENT CITY)

Scenario: A customer spent ₹50,000 in Delhi and later ₹30,000 in Bengaluru. Need sales by city at time of purchase and all historical sales by current city.

Table Design:

dim_customer (SCD Type 2)

customer_sk	customer_id	name	city	start_date	end_date	is_current
501	C101	Rahul	Delhi	2024-01-01	2024-06-30	N
502	C101	Rahul	Bengaluru	2024-07-01	9999-12-31	Y

fact_sales (order line level)

order_id	customer_sk	product_sk	date_sk	amount
01	501	101	2024-02-10	50000
02	502	101	2024-08-15	30000

Answer:

A. Sales by historical city (city at time of purchase)

```
SELECT c.city, SUM(f.amount) as total_sales
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_sk = c.customer_sk
GROUP BY c.city;
```

Result

city	total_sales
Delhi	50000
Bengaluru	30000

B. Sales by current city (all historical sales grouped by current city)

```
SELECT c_curr.city, SUM(f.amount) as total_sales
FROM fact_sales f
JOIN dim_customer c_hist
    ON f.customer_sk = c_hist.customer_sk
JOIN dim_customer c_curr
    ON c_hist.customer_id = c_curr.customer_id
    AND c_curr.is_current = 'Y'
GROUP BY c_curr.city;
```

Result

city	total_sales
Delhi	0
Bengaluru	80000

Key Points:

- Use surrogate key in fact table
- Maintain effective dates + current flag
- Join on customer_id for current info
- Handle late arriving data using transaction date

6. SCENARIO 2 – GRAIN & DOUBLE COUNTING

Scenario: An order has 3 products and ₹500 shipping charge. Need revenue by product, by order and total shipping revenue.

Tables:

fact_sales (order line level)

order_id	line_id	product_sk	qty	amount
0101	1	101	1	50000
0101	2	102	2	2000
0101	3	103	1	3000

fact_order (order level)

order_id	customer_sk	date_sk	shipping_amount
0101	501	20250101	500

Answer:

1. Use order line level fact for product-level analysis.
2. Keep shipping in separate order-level fact (or allocate).
3. Avoid repeating shipping on each line to prevent double counting.

Example Queries:

7. SCENARIO 3 – INFLATED REVENUE AFTER JOIN

Scenario: Revenue before join = ₹200 Cr, after joining with dim_customer = ₹350 Cr. No duplicate records in fact table.

Expected Reason:

- Joined fact with dim_customer using customer_id (natural key)
- SCD Type 2 dimension has multiple rows per customer_id
- Join multiplied fact rows → inflated revenue

How to Investigate:

1. Check join keys and cardinality.
2. Compare row counts before and after join.
3. Check for multiple matching dimension rows.
4. Validate SCD effective dates and current flag.
5. Ensure using customer_sk in fact table.

Correct Join:

```
SELECT SUM(f.amount)
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_sk = c.customer_sk;
```

Key Points:

- Use surrogate key (customer_sk)
- Check for duplicate dimension records / overlapping dates
- Don't use DISTINCT to hide the issue
- For many-to-many (e.g. multiple salespeople), use bridge table or allocation logic

8. SCENARIO 4 – CHOOSE FACT TABLE TYPE

Scenario: Need to report (A) Sales, (B) Inventory balance, (C) Order fulfillment.

Requirement	Fact Type	Grain	Example Columns
A. Sales	Transaction Fact	One row per order line	order_id, product_sk, qty, amount, date_sk
B. Inventory	Periodic Snapshot	One row per product per warehouse per day	product_sk, location_sk, date_sk, inventory_qty
C. Fulfillment	Accumulating Snapshot	One row per order	order_id, placed_date, shipped_date, delivered_date, status flags

Key Points:

- Use separate fact tables for different business processes
- Inventory balances are semi-additive (don't sum across time)
- Accumulate milestones for fulfillment
- Maintain consistent grain and clear business definition

9. SCENARIO 5 – STAR vs SNOWFLAKE DECISION

Scenario: 50M products with Category → Subcategory → Brand → Product. Need dashboards revenue by category, brand, product.

Answer:

- Start with Star Schema (denormalized dim_product) for simplicity and performance.
- Consider Snowflake if hierarchy is very complex or shared across multiple models.
- Both can handle SCD Type 2 if category changes over time.

Key Factors:

- Query performance, maintenance, hierarchy complexity, data volume.

10. SCENARIO 6 – DATA LAKE vs WAREHOUSE vs LAKEHOUSE

Scenario: 10 TB daily data (JSON logs, DB, PDFs, images). Finance needs revenue dashboards, DS needs raw data, AI team needs document search.

Answer: Use Lakehouse Architecture

- Bronze: raw ingestion (all data types)
- Silver: cleaned, validated, integrated
- Gold: facts, dimensions, aggregates for BI
- Supports BI, ML and AI use cases with governance.

Key Considerations:

- Data formats, consumers, latency, cost, governance, scalability
- If only structured finance reporting → a data warehouse may be enough.